

Delhi Air Pollution Analysis Using Python

Intermediate Level Task

Submitted By:

Venmathi P

College:

Rajalakshmi Institute of Technology

Department:

CSE

Date:

05/07/2026

Table of Contents

1. Abstract
2. Introduction
3. Objectives
4. Dataset Description
5. Research Questions
6. Software and Libraries Used
7. Data Exploration
8. Data Cleaning
9. Statistical Analysis
10. Data Visualization
11. Key Findings
12. Conclusion
13. References

1. Abstract

Air pollution is one of the major environmental challenges affecting public health worldwide. This project analyzes Delhi's air pollution dataset using Python. The analysis focuses on pollutant concentrations such as PM2.5, PM10, CO, NO, NO₂, O₃, SO₂, and NH₃. Statistical analysis and visualization techniques are used to identify pollution trends, understand relationships among pollutants, and gain insights into air quality. The project demonstrates how Python libraries such as Pandas, Matplotlib, Seaborn, and Plotly can be used for environmental data analysis.

2. Introduction

Air pollution has become a serious environmental issue, especially in metropolitan cities like Delhi. High concentrations of pollutants such as particulate matter and harmful gases negatively affect human health and the environment. Data analysis helps identify pollution patterns and supports informed decision-making. In this project, pollutant concentration data is analyzed using Python to understand trends and relationships between different air pollutants.

3. Objectives

The objectives of this project are:

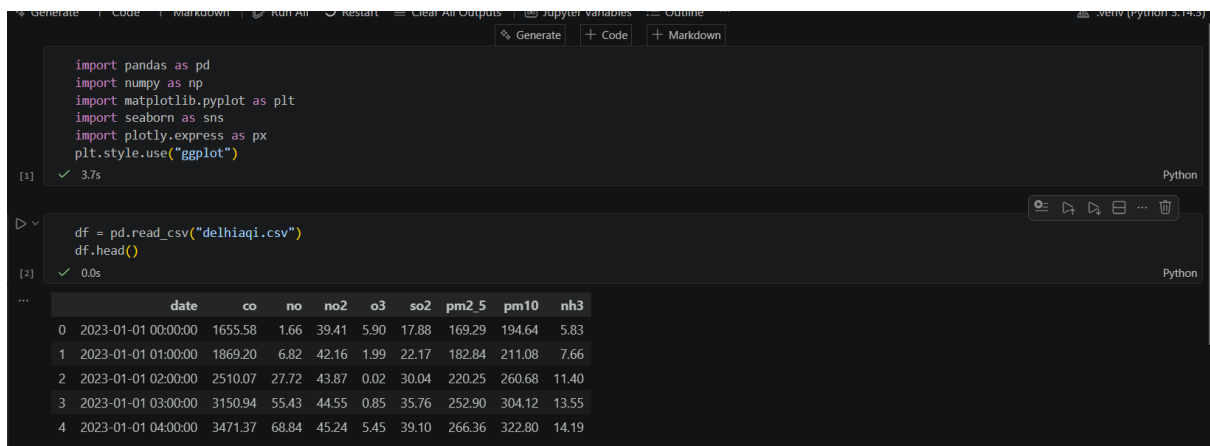
- To analyze Delhi air pollution data.
- To perform data cleaning and preprocessing.
- To study pollutant concentration trends.
- To visualize pollution data using different graphs.
- To perform statistical analysis.
- To identify relationships among pollutants.

4. Dataset Description

The dataset contains daily pollutant concentration measurements collected from Delhi.

The dataset consists of the following columns:

- Date
- Carbon Monoxide (CO)
- Nitric Oxide (NO)
- Nitrogen Dioxide (NO₂)
- Ozone (O₃)
- Sulfur Dioxide (SO₂)
- PM2.5
- PM10
- Ammonia (NH₃)



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
plt.style.use("ggplot")

[1] ✓ 3.7s Python

df = pd.read_csv("delhiaqi.csv")
df.head()
```

	date	co	no	no2	o3	so2	pm2_5	pm10	nh3
0	2023-01-01 00:00:00	1655.58	1.66	39.41	5.90	17.88	169.29	194.64	5.83
1	2023-01-01 01:00:00	1869.20	6.82	42.16	1.99	22.17	182.84	211.08	7.66
2	2023-01-01 02:00:00	2510.07	27.72	43.87	0.02	30.04	220.25	260.68	11.40
3	2023-01-01 03:00:00	3150.94	55.43	44.55	0.85	35.76	252.90	304.12	13.55
4	2023-01-01 04:00:00	3471.37	68.84	45.24	5.45	39.10	266.36	322.80	14.19

5. Research Questions

1. How does PM2.5 concentration change over time?
2. What is the distribution of PM2.5 values?
3. Is there a relationship between PM2.5 and PM10?
4. Which pollutant has the highest average concentration?
5. How are pollutants correlated with one another?

6. Software and Libraries Used

Software

- Visual Studio Code
- Python 3.x
- Jupyter Notebook

Python Libraries

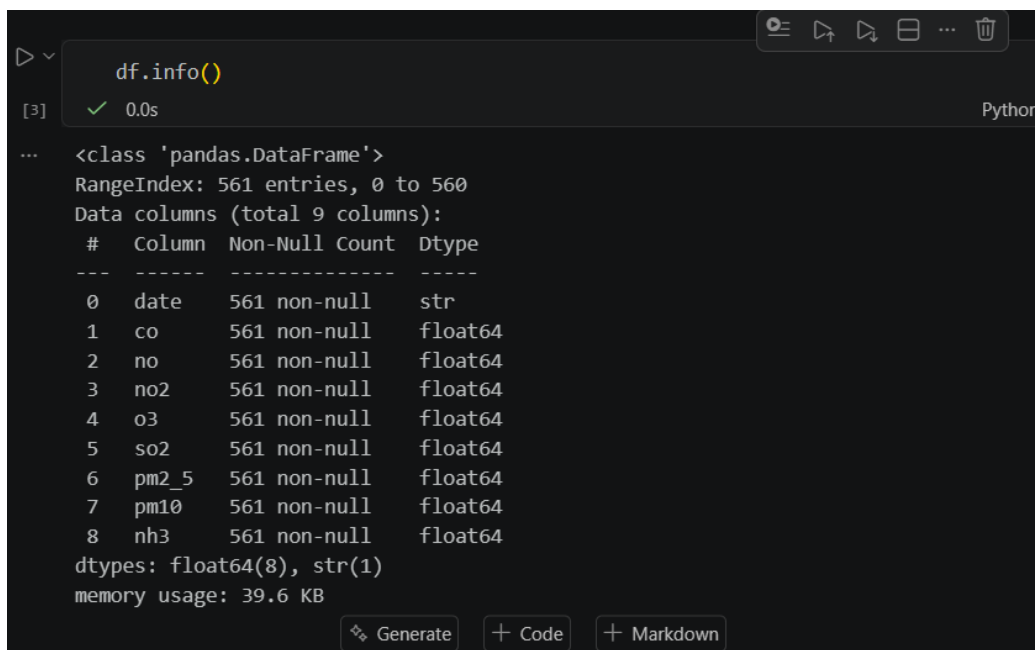
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Plotly

7. Data Exploration

Dataset Information

Briefly explain:

The dataset was explored to understand its structure, number of rows and columns, data types, and missing values



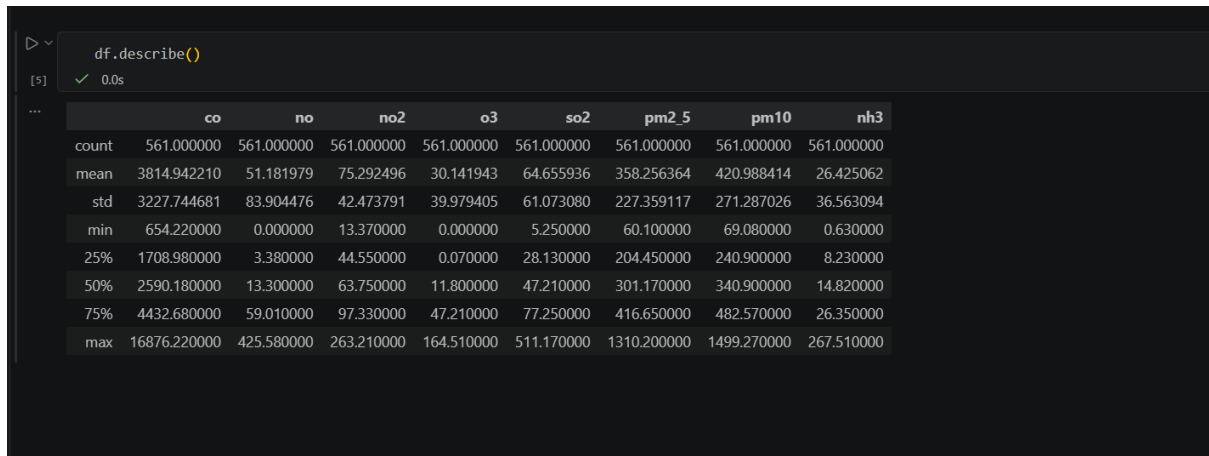
```
df.info()
[3] ✓ 0.0s Python

... <class 'pandas.DataFrame'>
RangeIndex: 561 entries, 0 to 560
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype  
---  -
0    date        561 non-null    str     
1    co           561 non-null    float64
2    no           561 non-null    float64
3    no2          561 non-null    float64
4    o3           561 non-null    float64
5    so2          561 non-null    float64
6    pm2_5        561 non-null    float64
7    pm10         561 non-null    float64
8    nh3          561 non-null    float64
dtypes: float64(8), str(1)
memory usage: 39.6 KB
```

Statistical Summary

Briefly explain:

Statistical measures such as mean, standard deviation, minimum, and maximum values were calculated to understand the distribution of pollutant concentrations.



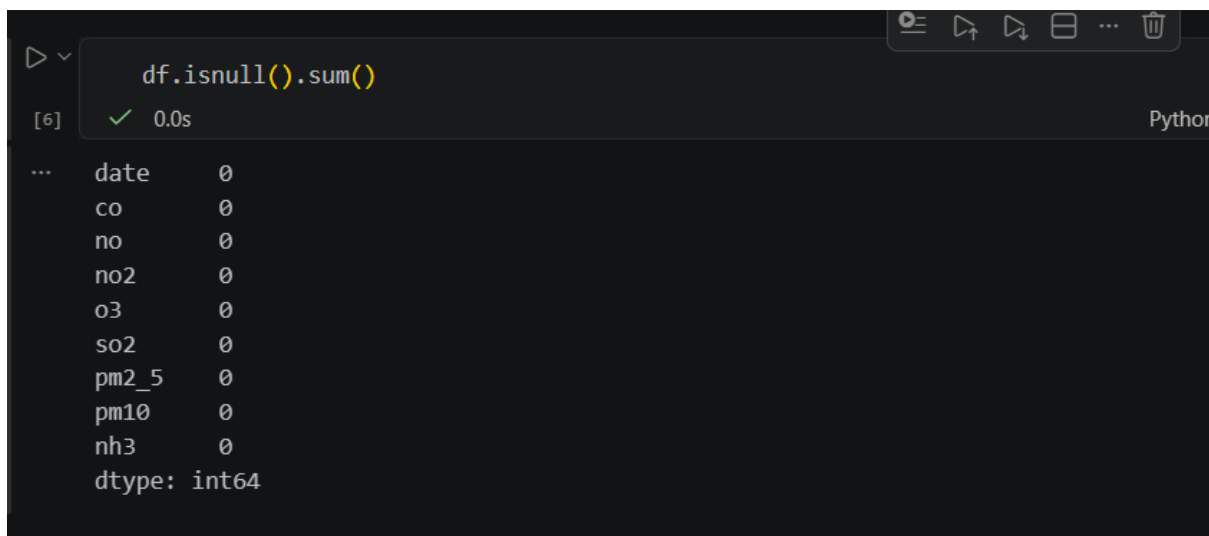
```
df.describe()
```

	co	no	no2	o3	so2	pm2_5	pm10	nh3
count	561.000000	561.000000	561.000000	561.000000	561.000000	561.000000	561.000000	561.000000
mean	3814.942210	51.181979	75.292496	30.141943	64.655936	358.256364	420.988414	26.425062
std	3227.744681	83.904476	42.473791	39.979405	61.073080	227.359117	271.287026	36.563094
min	654.220000	0.000000	13.370000	0.000000	5.250000	60.100000	69.080000	0.630000
25%	1708.980000	3.380000	44.550000	0.070000	28.130000	204.450000	240.900000	8.230000
50%	2590.180000	13.300000	63.750000	11.800000	47.210000	301.170000	340.900000	14.820000
75%	4432.680000	59.010000	97.330000	47.210000	77.250000	416.650000	482.570000	26.350000
max	16876.220000	425.580000	263.210000	164.510000	511.170000	1310.200000	1499.270000	267.510000

Missing Values

Briefly explain:

The dataset was checked for missing values before performing the analysis.



```
df.isnull().sum()
```

date	0
co	0
no	0
no2	0
o3	0
so2	0
pm2_5	0
pm10	0
nh3	0
dtype: int64	

8. Data Cleaning

The following preprocessing steps were performed:

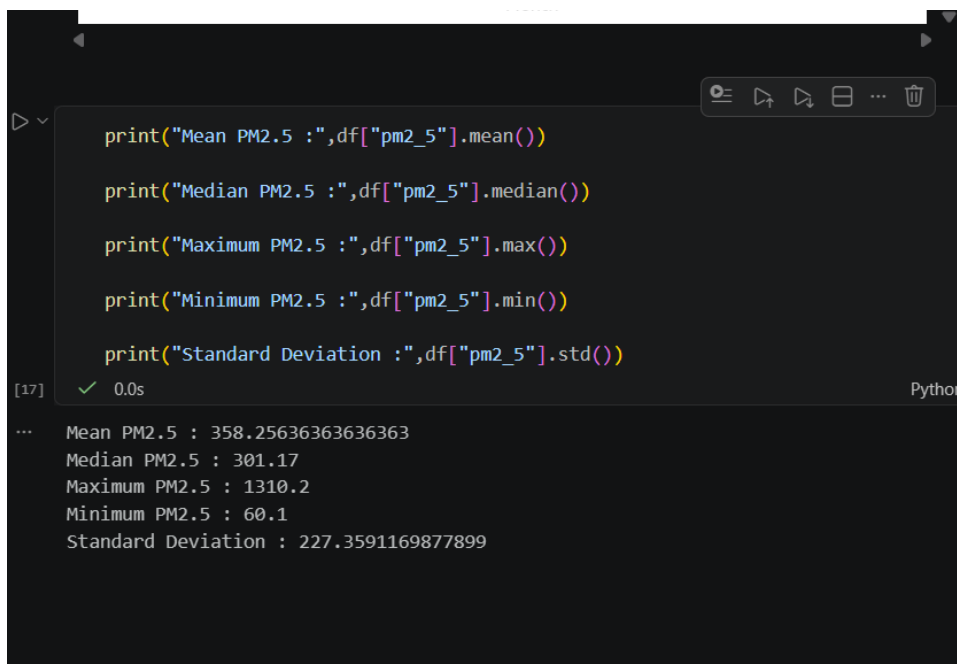
- Converted the date column into datetime format.
- Checked for missing values.
- Removed duplicate records (if any).

9. Statistical Analysis

Statistical analysis was performed on the PM2.5 values.

Include the output of:

- Mean
- Median
- Maximum
- Minimum
- Standard Deviation



```
print("Mean PM2.5 :",df["pm2_5"].mean())

print("Median PM2.5 :",df["pm2_5"].median())

print("Maximum PM2.5 :",df["pm2_5"].max())

print("Minimum PM2.5 :",df["pm2_5"].min())

print("Standard Deviation :",df["pm2_5"].std())
```

[17] ✓ 0.0s Python

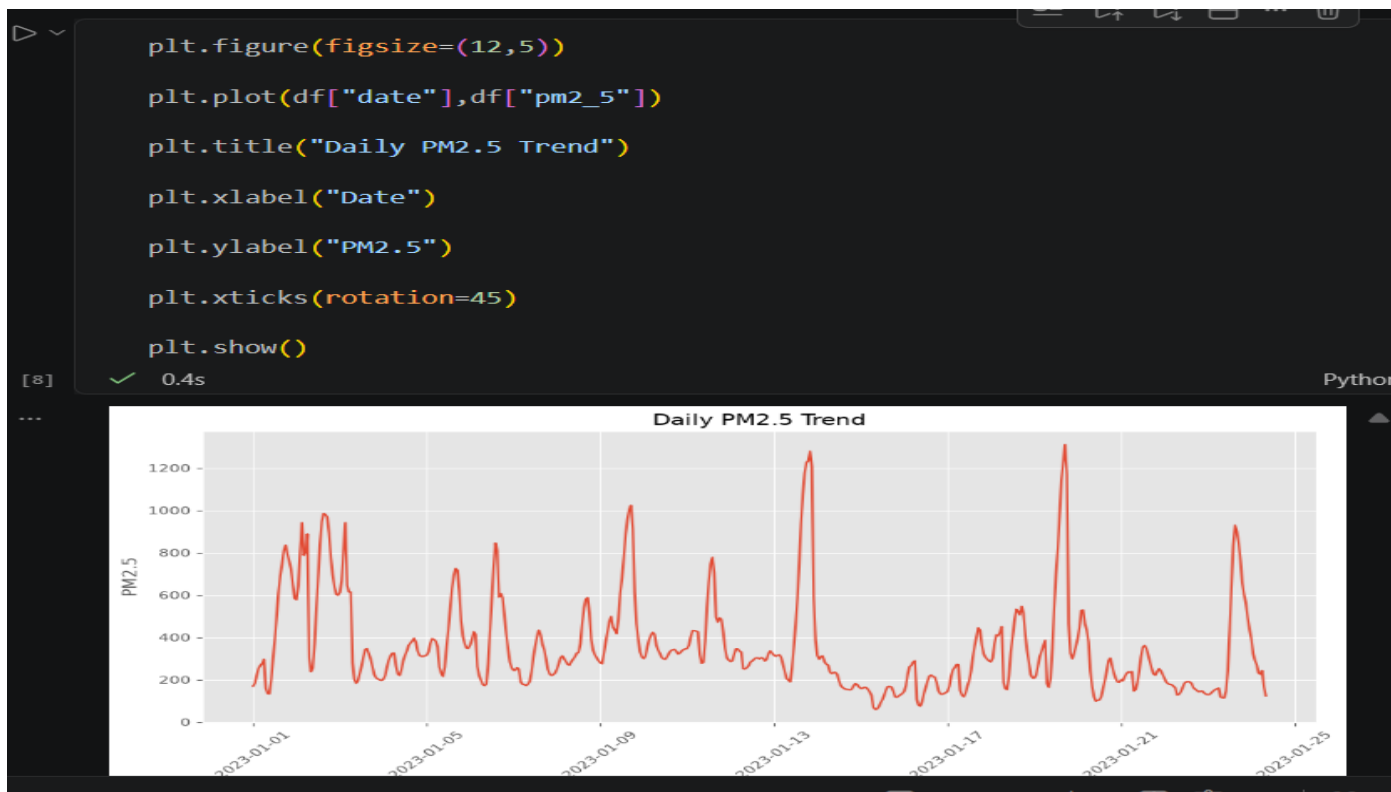
```
... Mean PM2.5 : 358.25636363636363
Median PM2.5 : 301.17
Maximum PM2.5 : 1310.2
Minimum PM2.5 : 60.1
Standard Deviation : 227.3591169877899
```

10. Data Visualization

Daily PM2.5 Trend

Description:

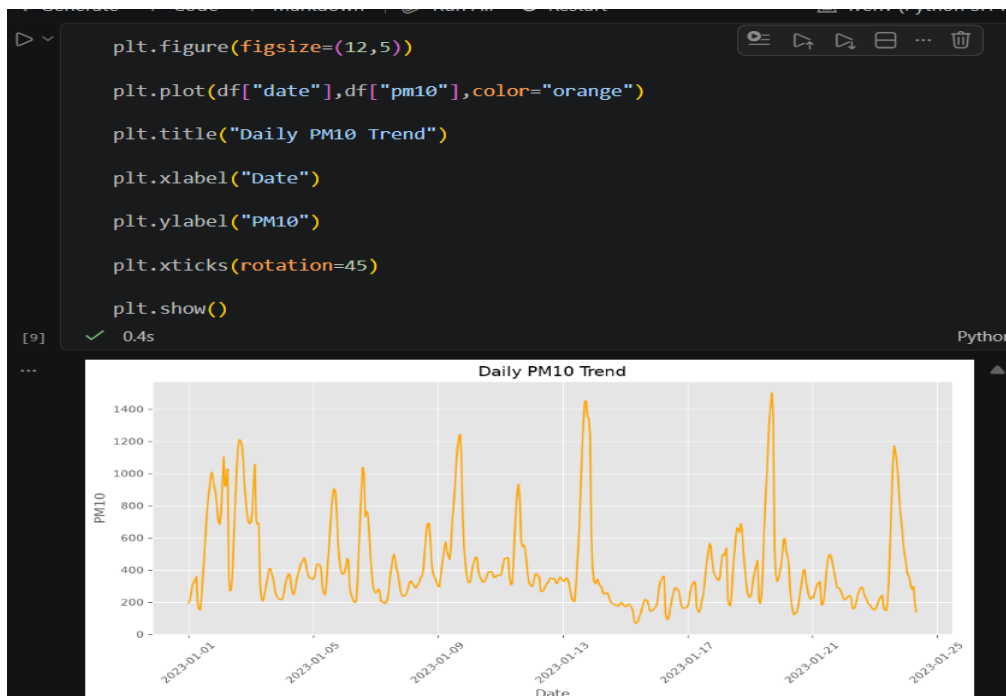
The graph shows the variation of PM2.5 concentration over time. It helps identify periods of higher and lower pollution levels.



Daily PM10 Trend

Description:

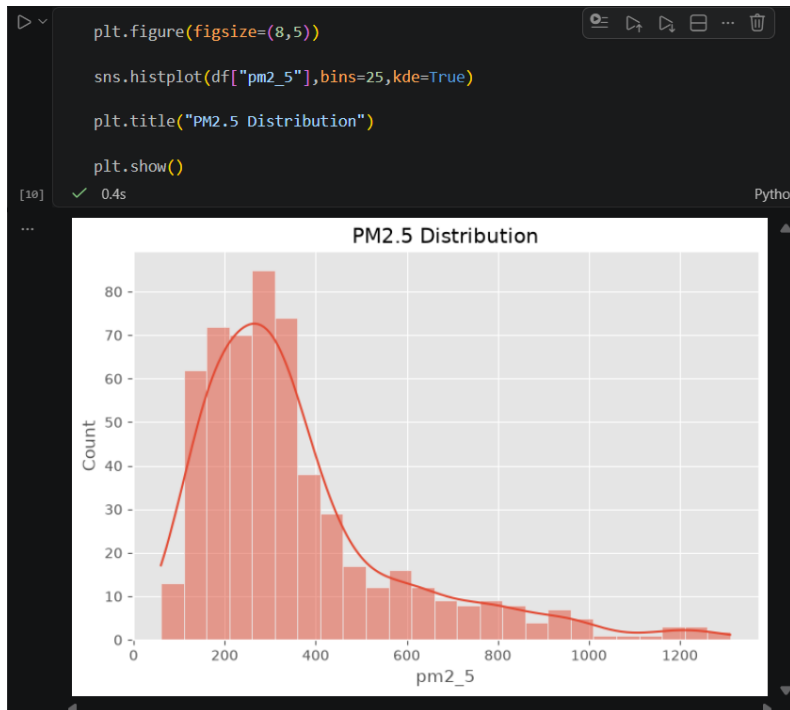
This graph illustrates changes in PM10 concentration over time, helping identify trends in coarse particulate matter.



PM2.5 Distribution

Description:

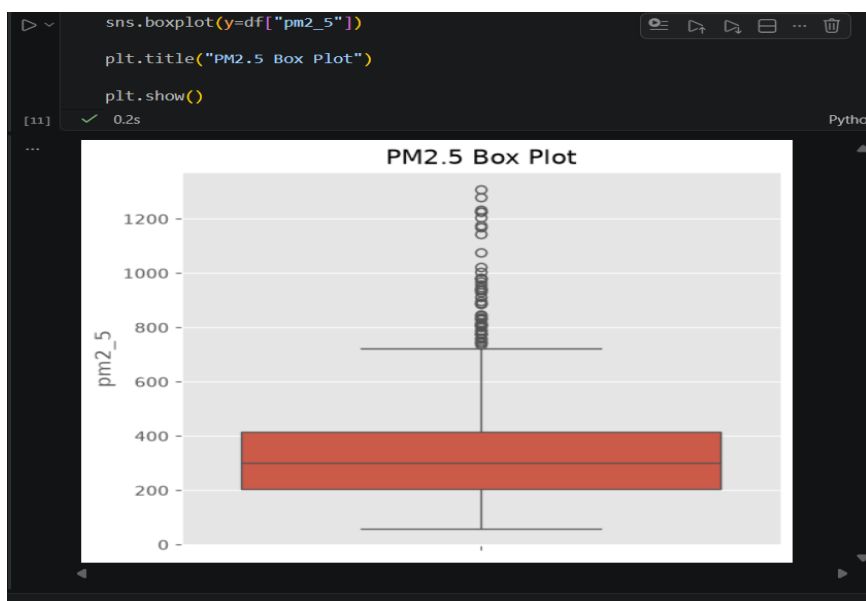
The histogram displays the frequency distribution of PM2.5 values and helps identify common concentration ranges.



PM2.5 Box Plot

Description:

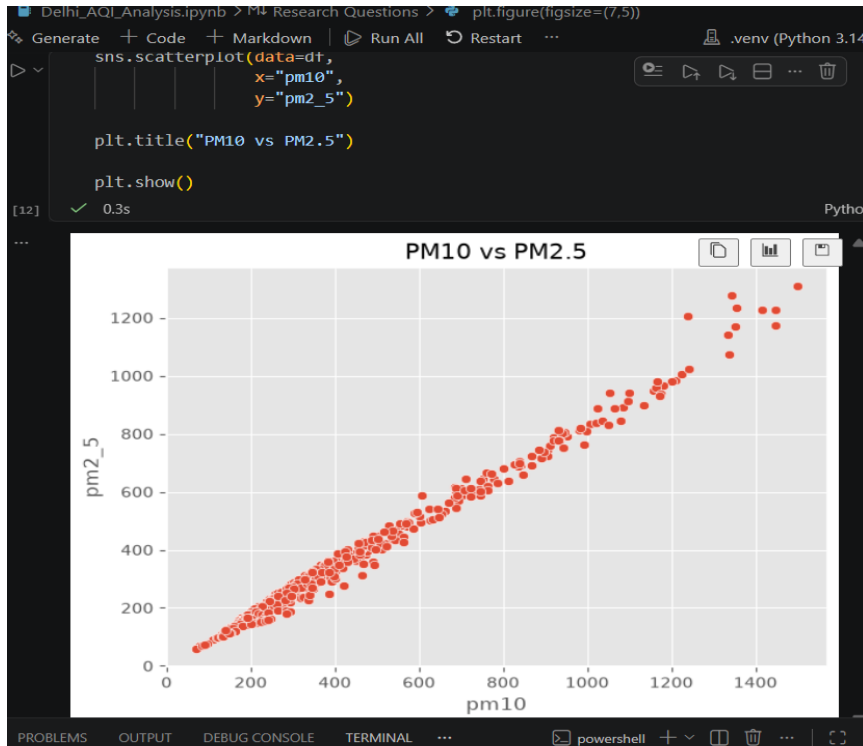
The box plot summarizes the distribution of PM2.5 and highlights outliers.



PM10 vs PM2.5 Scatter Plot

Description:

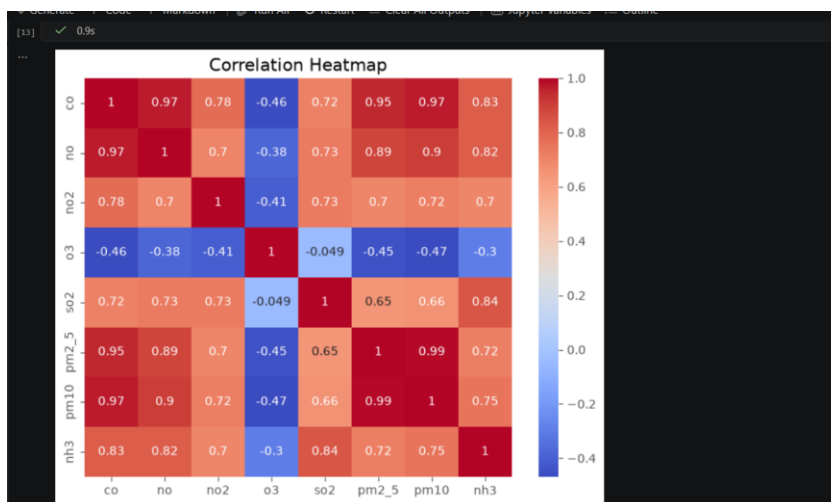
The scatter plot shows the relationship between PM10 and PM2.5 concentrations and helps identify any positive correlation.



Correlation Heatmap

Description:

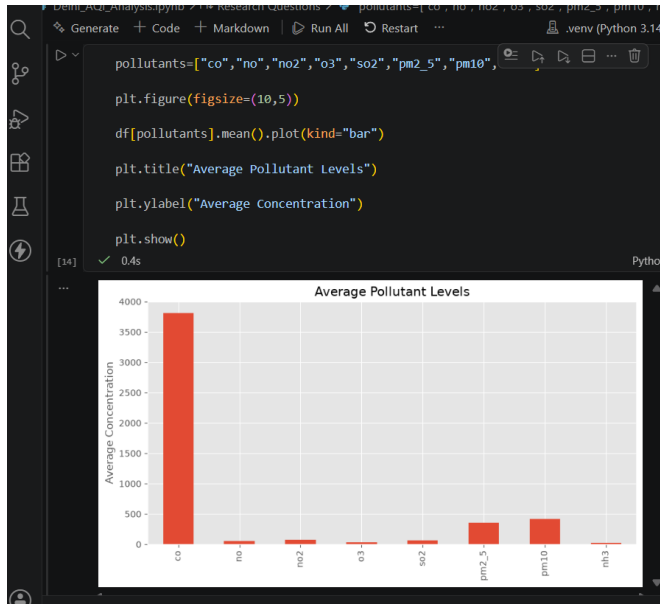
The heatmap visualizes the correlation among pollutants, making it easier to identify strong positive or negative relationships.



Average Pollutant Levels

Description:

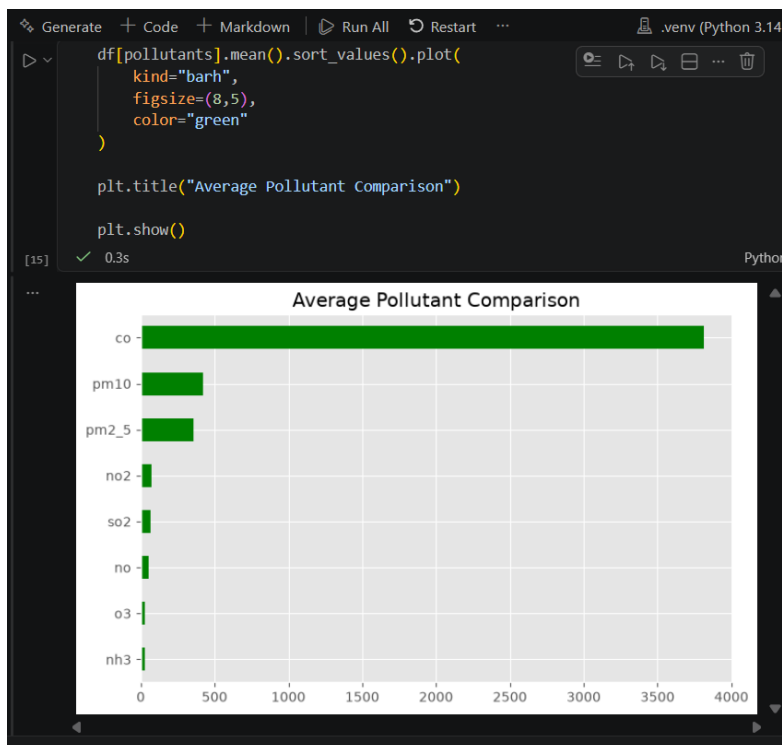
The bar chart compares the average concentrations of different pollutants.



Average Pollutant Comparison

Description:

This chart ranks pollutants according to their average concentration, providing a clear comparison.



Monthly Average PM2.5

Description:

The graph displays the average PM2.5 concentration for each month, highlighting seasonal variations.



Interactive PM2.5 Trend

Description:

An interactive Plotly graph provides a dynamic view of PM2.5 changes over time.



11. Key Findings

- PM2.5 levels fluctuate over time, indicating changes in pollution intensity.
- PM10 and PM2.5 show a positive relationship.
- The heatmap reveals correlations among several pollutants.
- PM2.5 and PM10 are significant contributors to poor air quality.
- Data visualization makes it easier to understand pollutant patterns.

12. Conclusion

This project successfully analyzed Delhi's air pollution dataset using Python. Various visualization techniques and statistical methods were used to study pollutant concentrations and their relationships. The analysis shows that particulate matter, especially PM2.5 and PM10, plays a significant role in air pollution. Such analyses can support environmental monitoring and informed decision-making to improve air quality.